COS10022 Assignment 1 — Report Guide

What theory and content your report needs

Study notes generated with Claude — do not paste into your report

How to use this guide

This is a checklist of what each part of the report must contain, plus the theory behind it. It is written as notes *to you*. Write the report yourself, in your own words, from your own KNIME run, in `report_template.tex`. Declare any genAI use (including this guide) in the appendix. The week 1 slides require prompts and outputs to be acknowledged.

Hard requirements from the brief

- First page top: name, student ID, unit code, class (CL1/0x), tutor's name.
- At most 10 pages; mostly typed text (not screenshots of text); no table of contents needed.
- Answer the questions *in the order of the brief*: Task 1 then Task 2.
- Turnitin below 25% overall and below 5% from any single source.
- Zip: workflow + input files in the workflow directory + output/intermediate files + video. The workflow must run and give the same numbers as the report.

Suggested page budget (10 pages max)

Section	Pages
Introduction + dataset overview (short)	0.5
Task 1: workflow, node configuration, loading issues, results	2.5
Task 2: input, splitter, Workflow Coach, partitioning, models	3.5
Task 2: results, comparison, insights, suggestions	2
Exploration / reflection + conclusion	1
Appendix: genAI declaration (may sit after the 10 pages if your tutor allows; ask)	—

1 Theory you need to understand

1.1 Where this sits in the Data Analytics Lifecycle

The assignment covers Phase 3 (Model Planning: choosing LR, NB, RF) and Phase 4 (Model Building: training on one dataset, evaluating on a separate test dataset). The lectures stress *documenting every decision* in Phase 4. That is the reason the rubric gives 15% to node configuration.

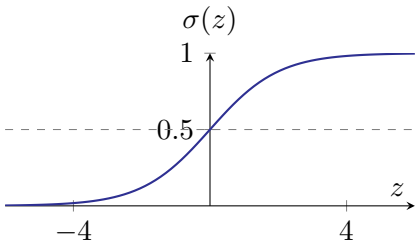
1.2 Supervised classification

Inputs: Age, Gender, Speed_of_Impact, Helmet_Used, Seatbelt_Used. Output (class label): Survived = Yes/No. The training set teaches the model; the test set estimates performance on unseen data. The lectures say training and test sets should be *independent and non-overlapping*. Keep that sentence in mind for the leakage discussion in Task 2.

1.3 Logistic Regression

A linear combination of the inputs, $z = w_0 + w_1x_1 + \cdots + w_kx_k$, is passed through the sigmoid:

$$P(\text{Survived} = \text{Yes} \mid x) \;=\; \sigma(z) \;=\; \frac{1}{1 + e^{-z}}$$



- The output is a probability in (0,1). Predict "Yes" if it is above the threshold (0.5 by default).
- Equivalently, $\log \frac{p}{1-p} = z$: each coefficient is the change in the *log-odds* for a one-unit increase in that input. e^w is the odds ratio.
- Nominal inputs (Gender, Helmet, Seatbelt) are dummy-coded; the coefficient is relative to the base level.
- **Reference category** (KNIME setting): coefficients describe the non-reference class against the reference. With reference = No, a positive coefficient means "raises the odds of surviving". State which reference you used, or readers can't interpret your signs.
- **Solver** (KNIME setting): IRLS (iteratively reweighted least squares) gives the exact maximum-likelihood solution and is deterministic. SAG (stochastic average gradient) is approximate, depends on epochs, step size and seed, and suits big data. Be able to justify your choice.
- The decision boundary is linear in the inputs. It cannot represent a sharp cut-off such as "speed $\geq 90 \Rightarrow$ always dies" except as a smooth slope. Expect this to limit LR.

1.4 Naïve Bayes

$$P(c \mid a_1, \ldots, a_m) \;\propto\; P(c) \prod_{j=1}^{m} P(a_j \mid c)$$

- "Naïve" = assumes the attributes are independent given the class.
- Nominal attributes: frequency tables. Numeric attributes (Age, Speed): KNIME fits a Gaussian per class. A Gaussian is a poor fit for a hard threshold at speed 90. Worth mentioning.
- $P(c)$ is the **prior**, learned from the class proportions in the training set. Your training set is 50/50 by construction, but the test set is about 94/6. The model's prior is therefore wrong for the test data, so it over-predicts "Yes". This is a strong insight point.
- Lecture advantages: simple, fast, needs little data. Disadvantage: the independence assumption.

1.5 Random Forest

- An ensemble of decision trees. Each tree is trained on a *bootstrap sample* (rows drawn with replacement) and considers a random subset of attributes at each split ($\sqrt{k}$ by default). The final prediction is the majority vote.
- Trees split on thresholds, so they can learn "speed ≥ 90" directly. That is non-linear behaviour LR can't express.
- Deep, unpruned trees can *memorise* individual records. When test records also appear in the training data (see leakage below), RF benefits the most.
- Its result depends on a random seed. Fix it for reproducibility.

1.6 Evaluation metrics (week 4 lecture)

Confusion matrix with positive class = Yes:

	Pred. Yes	Pred. No
Actual Yes	TP	FN
Actual No	FP	TN

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad \text{Recall (TPR, sensitivity)} = \frac{TP}{TP+FN} \quad \text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Specificity (TNR)} = \frac{TN}{TN+FP} \qquad F_1 = \frac{2 \cdot P \cdot R}{P+R} \qquad \text{Cohen's } \kappa = \frac{p_o - p_e}{1-p_e}$$

- KNIME's Scorer shows all of these per class (Accuracy statistics port) plus Cohen's kappa.
- **Always compare against the majority-class baseline**: a model that always says "No". If the baseline is 94%, an 85% model is *worse at accuracy* than doing nothing, even if it finds survivors. This is the core of the imbalance discussion.
- The lecture says a good model has accuracy of 80% or more, *but accuracy alone is not enough.* Use that line to motivate recall, precision and kappa.

1.7 IID, sampling, reproducibility, leakage

- **IID** = independent and identically distributed: each record is drawn independently from the same distribution. The brief calls the balanced 700 set "IID". Think critically: a set with 50/50 classes no longer has the class distribution of the original data or of the test set. That is worth one or two sentences of reflection (rubric: critical thinking).
- **Random vs. top-k sampling**: the files are sorted, so "take from top" gives a biased sample.
- **Seeds**: Partitioning, Shuffle and RF are random. Without fixed seeds a re-run gives different numbers, and the brief says a mismatch with the report gets no mark.
- **Data leakage**: test records also present in the training set inflate test scores, because the model is partly "tested" on data it has already seen.

2 Dataset facts

Facts from the data (check them in your own KNIME run)

- 6 columns: Age (int, 18–69), Gender (Male/Female), Speed_of_Impact (int, 20–119), Helmet_Used (Yes/No), Seatbelt_Used (Yes/No), Survived (Yes/No). No missing values.

- train1/train2: 420 rows each; test1/test2 (.csv): 180 rows each; test2.txt = test2.csv but **tab-separated with CRLF line endings**.
- train1+test1 and train2+test2 are **two different 70/30 splits of the same 600 records**. Reading all four CSVs gives 1,200 rows with every record twice (599 distinct).
- Files are sorted: all "No" rows first, then "Yes", each in descending speed.
- All 4 CSVs: 378 Yes / 822 No. After the split: train 350 Yes + 350 No = 700; test 28 Yes + 472 No = 500.
- Speed ≥ 90: survival 0%. Speed < 90: roughly 45% survive. Age has almost no effect. Helmet and seatbelt raise survival. You can show this in KNIME with a GroupBy or chart: that counts as exploration.

3 Task 1 — what your section must contain

Brief: explain how you came up with the workflow, the detailed configurations and why; describe your prediction results; describe difficulties loading the files and how you solved them.

Rubric hooks: Workflow design 5%, Node configuration 15%, Data loading 10%, Evaluation 10%, Troubleshooting 5%

1. **Workflow overview**: one screenshot of the Task 1 part, then 3–5 sentences on the logic (read train, read test, learn, predict, score) and *why* the order matters.
2. **Node configuration table**: one row per node with setting, value and *justification*. Justification is what earns the 15%. Include at least:
   - CSV Reader (train): file, delimiter, header row, detected column types.
   - CSV Reader (test): delimiter = Tab, and why.
   - LR Learner: target column, included features, reference category, solver, max epochs/epsilon.
   - LR Predictor: whether probabilities are appended.
   - Scorer: actual vs. predicted column.
3. **Loading difficulty**: what you saw (the .txt read as one column), why (tab delimiter, CRLF), how you fixed it, how you confirmed the fix (6 columns, 180 rows, types match the training file). Screenshot before and after. Also mention that train and test must have identical column names and types for the predictor to work.
4. **Results**: confusion matrix screenshot and accuracy statistics. Then interpret:
   - accuracy compared with the always-"No" baseline for this test set (111/180 = 61.7%);
   - recall for "Yes" (how many survivors it finds) compared with recall for "No";
   - the coefficient table: which features increase or decrease survival odds, which are significant (p-value), and whether that makes sense (seatbelt, helmet, speed, age).

Trap

Expected Task 1 accuracy with IRLS: 64.4% (No: 92 right/19 wrong; Yes: 45 wrong/24 right). If yours differs a lot, check the delimiter and column types before writing anything.

4 Task 2 — what your section must contain

Brief: how you took in all CSV files; the rules in the splitter; the prompt used to find the partitioning node; detailed configuration of the data-preparation nodes; issues during training and solutions; compare the three models; insights and suggestions.

4.1 Data preparation

Rubric hooks: Rule-based Row Splitter 5%, Partitioning 10%, Workflow Coach 5%, Node configuration 15%

- **Reading all CSVs**: CSV Reader in *Files in folder* mode on `src`, with an extension filter for `csv`. Explain why: the brief forbids changing the src folder, and the .txt must be excluded. Mention the options you checked: RowID handling across files, and *not* appending a path column (it would become a meaningless feature). Report the row count (1,200).
- **Splitter rule**: the exact rule text, which port holds which class, and the counts (378/822). Explain the rule syntax (`$column$`, `=>`, TRUE/FALSE).
- **Workflow Coach**: the *exact prompt you typed*, a screenshot, which nodes it suggested, and why you picked Partitioning. Relate the prompt to the week 2 prompt structure (task, context, constraints). Write and try your own prompt; that is what is graded.
- **Partitioning (both)**: Absolute 350, Draw randomly, seed, and why "random" (sorted files) and why a seed (reproducibility). Counts: 350/28 and 350/472.
- **Concatenate (both)**: which ports feed which node; how duplicate RowIDs are handled; result 700 and 500.
- **Shuffle (both)**: why shuffle (order-dependent learners, removes the class-sorted order), seed.

4.2 Models

Rubric hooks: Model construction 5%, Troubleshooting 5%

- One configuration table per model (or one combined table): target, features, key parameters, seed, with justification. Confirm all three use the *same* shuffled train and test nodes.
- Issues during training: report *what actually happened to you* (error or warning text, cause, fix). Common ones: target column not nominal or not selected, a predictor fed the wrong port, Scorer comparing the wrong columns, results changing on every run (no seed), the NB nominal-values limit.

4.3 Results, comparison, insights

Rubric hooks: Evaluation 10%, Comparison & insights 10%

1. One comparison table: accuracy, Cohen's κ, recall/precision/F_1 for "Yes", recall for "No", for LR, NB and RF. Add the always-"No" baseline (472/500 = 94.4%) as its own row.

2. Confusion-matrix screenshots for all three.
3. Points to discuss (use your own numbers):
    - **Imbalance**: all three models sit below the 94.4% baseline on accuracy but find most of the 28 survivors. Discuss which metric matters for this problem, e.g. the cost of predicting "survives" for someone who won't (lecture: Type I vs. Type II error).
    - **Prior shift**: trained on 50/50, tested on 94/6, so the models over-predict "Yes" (low precision). NB is affected directly through $P(c)$.
    - **Leakage**: about 44% of test rows are identical to training rows (every record appears twice in the 4 CSVs). In the reference run RF scored 0.79 on those rows and 0.72 on unseen ones, while LR and NB showed no gap. Leakage favours the model that can memorise, but measure the size of the effect in your own run rather than assuming it. You can show this in KNIME (e.g. a Joiner between test and train, or a Duplicate Row Filter run).
    - **Model behaviour**: RF can learn the speed-90 threshold; LR fits a smooth slope; NB's Gaussian and independence assumptions don't match that shape.
    - Compare Task 1 LR with Task 2 LR, and explain why the numbers differ (different training distribution and test set).
4. **Suggestions**: remove duplicates (Duplicate Row Filter) before splitting; use stratified sampling or k-fold cross-validation (X-Partitioner/X-Aggregator, lecture week 4); evaluate with kappa, F_1 or AUC instead of accuracy; tune the decision threshold; collect more informative features.

5 Exploration & self-learning (10%) and reflection

Pick one or two small experiments and *run them yourself*. Report what you expected, what happened and what you learned:

- Re-run Task 2 after a Duplicate Row Filter: how much do the RF numbers drop?
- LR solver: IRLS vs. SAG. Do the coefficients and accuracy change, and why?
- Change the seed 2–3 times: how stable are the results? That shows the variance of a single holdout split.
- Explore the speed threshold with a chart, linking data understanding to model choice.

The reflection should be specific: what surprised you, what you would do differently, how the theory (prior, leakage, imbalance) explained something you saw.

6 Report quality (10%)

- Number and caption every figure and table, and refer to each in the text ("Figure 3 shows . . . ").
- Screenshots must be readable: crop to the relevant node or dialog.
- Use short paragraphs, consistent terms ("training set", not three different names), and a spell check (genAI grammar help is an allowed use if declared).
- Cite the lecture or textbook (Dietrich 2015, EMC) when you state theory, so the theory is attributed and not copied.

7 GenAI appendix

List each tool, what you used it for, the prompts and the outputs (or excerpts), and what you changed or verified yourself. Include: the KNIME Workflow Coach/K-AI prompts (part of the task), any use of this guide or the reference workflow, and any grammar help.

8 Video checklist

Chain of thought (how you planned the workflow), the challenges (delimiter, reading the folder, exact 350 split, reproducibility, leakage) and how you solved them, and your takeaways. Show the workflow running. Talk from your own understanding; the report and video should tell the same story.